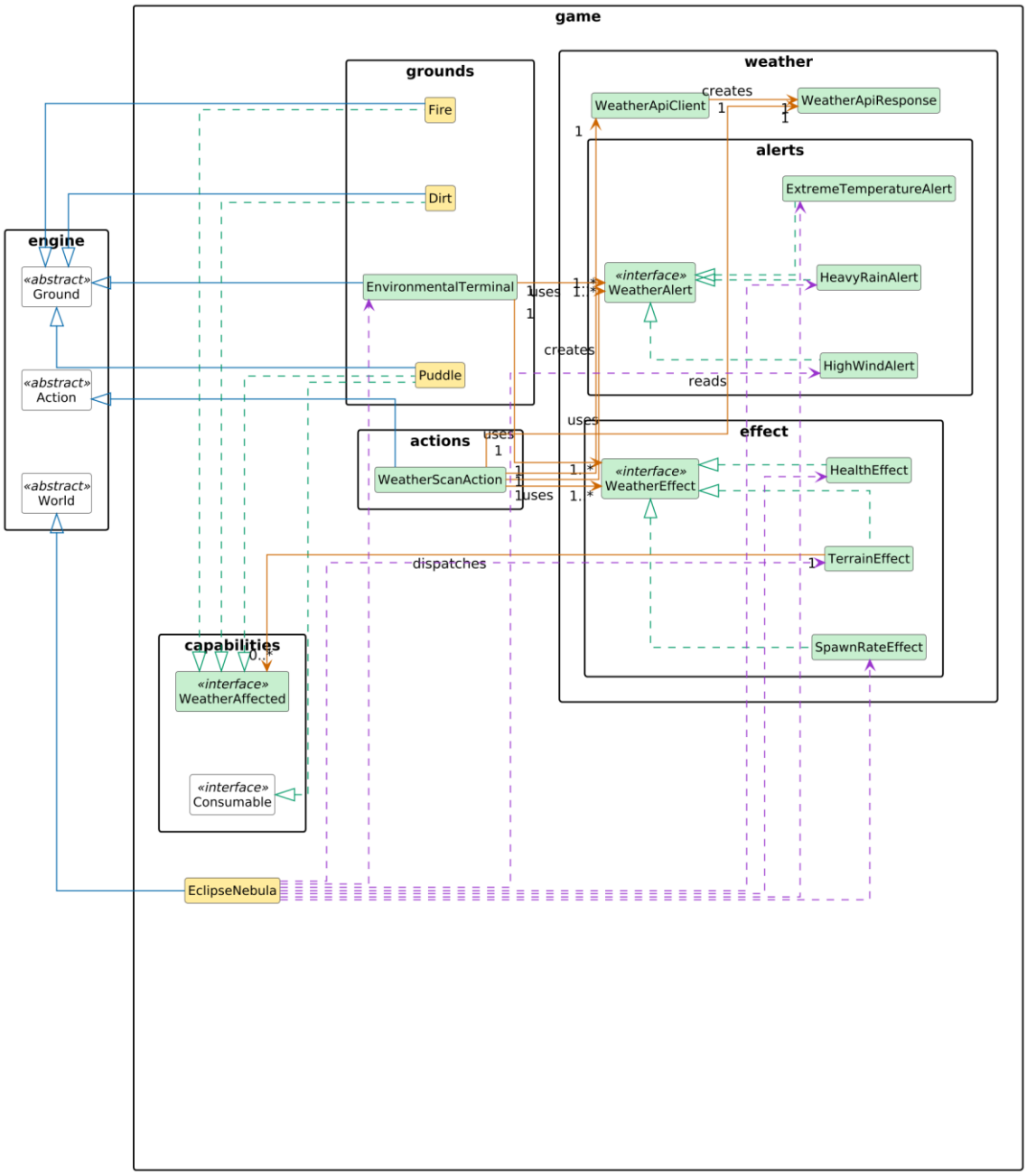
FIT2099 A3 REQ5 Design Document

UML Class Diagram



Design Rational

Design 1: Using abstract base classes to handle map-iteration, so you only have to write the code for what each specific actor does.

Pros	Cons
Removes duplication. HealthEffect, ExtremeTemperatureAlert and the two fire-iterating alerts all walk getXRange() and getYRange() and skip HAS_STERILISATION actors. Extracting this into an abstract MapWideActorEffect base class enforces DRY across several loops.	Forces single inheritance, where an effect that needs both actor-iteration and tile-iteration semantics (e.g. a "lightning strike" that hurts actors and ignites ground) can only extend one base, breaking the abstraction's contract or duplicating logic.
Sterilisation immunity is enforced in one place instead of three, eliminating a real bug class - currently the check must be remembered by every contributor adding a new temperature-based effect.	The abstract base becomes somewhat of a "god class" for weather concerns - any new iteration shape (radius, adjacent-only, line-of-sight) either bloats the base or requires another abstract class, splitting the hierarchy.
Subclasses become small (one applyTo(actor) method instead of full loops), improving readability and KISS.	Connascence of position increases and subclass overrides depend on the exact parameter order and lifecycle defined by the abstract template, and hence any changes to the base will affect its concrete classes.

Design 2: Replacing the WeatherCondition enum with a class system, where each condition (Rain, Snow, Clear) is its own class encapsulating its own reactions to a WeatherApiResponse.

Pros	Cons
Follows "Tell, Don't Ask" instead of HeavyRainAlert reading raw humidity and condition fields then re-deriving meaning, callers ask condition.extinguishes(weather) directly. Removes the connascence of meaning that the current Ability.EXTINGUISH marker tries to cover.	Combined condition with effect where Rain would now know how to mutate the map, blurring the clean DIP separation between data (condition), effect (WeatherEffect) and consequence (WeatherAlert) that is asked.
Adding a new condition (e.g., AcidStorm) becomes a single new class without touching any effect or alert, demonstrating OCP and future-proofing the system.	Every condition is forced to define every reaction method, even when irrelevant, a 13-constant enum becomes a 13-class hierarchy with mostly-empty overrides, violating ISP.
Eliminates the parallel Ability.EXTINGUISH marker entirely, since condition behavior is intrinsic to the class - fewer cross-cutting markers to maintain.	The JSON-to-condition mapping gets harder - valueOf(upper) works for an enum but not for a class, requiring a factory just to convert the API response to usable data.

Design 3: Small interfaces (WeatherEffect, WeatherAlert, WeatherAffected) consuming an immutable WeatherApiResponse record, where each concrete implementation reads only the fields it needs and branches on them directly.

Pros	Cons
Maintains the interface-driven structure already established by Assignment 2 (Consumable, Buyable, Sellable, Unlockable). Everything uses the same sort of build: small interface + wiring + concrete class.	Map-iteration code is duplicated across five methods, accepting some repetition as the cost of independence between effects.
Three abstractions and three concretions each satisfies REQ5 explicitly, and the higher-level classes (WeatherScanAction, EnvironmentalTerminal) depend only on the interfaces, enforcing DIP.	Sterilisation immunity must be remembered by each new actor-mutating effect - there is no compiler-enforced check, only convention.
The WeatherApiResponse record is immutable and easily constructable for tests, where no specific imitation build needed for the data carrier, only for the surfaces, which keeps unit tests fast and less prone to errors.	The WeatherAffected ground interface and TerrainEffect dispatcher are slightly redundant for a single-call use case. Could be made inline, but kept for codebase clarity with the other two effect types.
Adding a fourth effect or alert is one new class plus one line in the system, with no changes anywhere else. This demonstrates ease for open extension, and hence encapsulated and closed for modification.	

Summary

The implementation of REQ5 follows Design 3.

The aim for this requirement was to mirror the interface-driven structure already used throughout the codebase (Consumable, Buyable, Unlockable) rather than introducing a competing abstraction style mid-project. By keeping WeatherEffect, WeatherAlert and WeatherAffected as small interfaces, each concrete class declares its own thresholds and reactions individually, so adding HealthEffect does not constrain SpawnRateEffect, and adding HeavyRainAlert does not pre-judge what HighWindAlert is allowed to do.

To compare to the other designs 1 & 2:

> *"The abstract base becomes somewhat of a "god class" for weather concerns - any new iteration shape (radius, adjacent-only, line-of-sight) either bloats the base or requires another abstract class, splitting the hierarchy."*

Design 1 trades present duplication for future rigidity. The moment a new effect breaks the map-wide-iteration system, the template falls apart, and we end up with multiple abstract bases doing similar things. The duplication in Design 3 is real but mitigated; the rigidity Design 1 would introduce is unbounded.

> *" Combined condition with effect where Rain would now know how to mutate the map, blurring the clean DIP separation between data (condition), effect (WeatherEffect) and consequence (WeatherAlert) that is asked."*

Design 2 is the more natural object orientation option and would be reasonable in isolation, but the REQ5 specifically states that it requires three independent abstractions with three concretions each. Hence, collapsing WeatherCondition into a behavior-bearing hierarchy would actively undermine the rubric while making the JSON converting noticeably harder.

This is why I consider the usage of small interfaces + a record to wrap API data as the best design choice for this requirement, allowing the weather system to scale by adding classes rather than rewriting systems, while maintaining single responsibility (SOLID) and loose coupling between effects, alerts and terrain reactions.